

DANMAKU GAME PROJECT

弾幕ゲーム 改造マニュアル

オブジェクト指向プログラミングを知らなくても、
安心してコードに触れるようになるための手引きです。
「どこを」「どう変えると」「どうなるか」を具体的に示しま
す。

Processing (Java モー
ド)

初心者向
け

v1.0

第1章 このマニュアルについて	P.3
対象読者・ゲーム概要・開き方	
第2章 覚えておきたい5つの言葉	P.4
クラス・変数・メソッドなどをたとえ話で理解する	
第3章 ファイルの地図	P.6
どこに何が書いてあるかの一覧表	
第4章 よく使う改造レシピ集	P.8
目的別に「どこをどう変えるか」を具体的に解説	
第5章 やってはいけないこと	P.16
壊れやすいポイントと避け方	
第6章 困ったときは（エラー対処）	P.18
よくあるエラーと直し方	
第7章 用語集	P.20
出てきた言葉のまとめ	

1 このマニュアルについて

このマニュアルの対象読者

「クラス」「継承」「インスタンス」といった言葉に自信がなくても大丈夫です。このマニュアルは、プログラミング経験はあるけれど**オブジェクト指向の考え方に慣れていない**チームメンバーが、このゲームのコードを安心して改造・拡張できるようになることを目的としています。

このマニュアルでできるようになること

- ・敵の強さやプレイヤーの強さなどの「数値」を安全に調整できる
- ・弾や敵の色・見た目を変更できる
- ・新しい攻撃パターンや武器を、既存のコードを真似して追加できる
- ・エラーが出たときに、慌てず自分で直せる

このゲームはどんなゲームか

画面上部に固定された敵が弾幕（大量の弾）を撃ち続け、画面下部のプレイヤーがWASDキーで避けながら自動攻撃で敵弾を相殺し、制限時間まで生き残るゲームです。今後の機能追加を前提とした「土台」として作られています。

ファイルの開き方

- 1 配布されたフォルダを Processing（processing.org の無料アプリ）で開きます。
- 2 フォルダ内の `.pde` ファイルをダブルクリックすると Processing が起動します。
- 3 ▶（実行）ボタンを押すとゲームが動きます。まず一度動かして遊んでみましょう。
- 4 コードを直すときは、Processingのエディタ内でそのまま文字を書き換えて保存し、再度▶を押します。

はじめに一度、動かしてみましょう

改造前に一度動作を確認しておくで、「何がどう変わったか」を体感で理解しやすくなります。何かおかしくなったら、配布時点のファイルと見比べれば元に戻せます。

覚えておきたい5つの言葉

このコードには「オブジェクト指向」という考え方が使われています。難しく聞こえますが、要は「役割ごとに部品を分けて、部品同士を組み合わせでゲームを作る」という工夫です。まずは5つの言葉だけ、たとえ話でざっくり覚えておきましょう。

クラス (class)



「設計図」のことです。例えば `Enemy` というクラスは「敵とはこういうものだ」という設計図。設計図そのものは画面には出てきません。

インスタンス (オブジェクト)



設計図（クラス）をもとに実際に作られた「モノ」のことです。 `new Enemy(...)` と書くと、設計図から実物の敵が1体作られます。たい焼きの「型」がクラス、実際に焼き上がった「たい焼き」がインスタンスだとイメージしてください。

変数 (へんすう)



値を入れておく「箱」です。 `float PLAYER_SPEED = 4.0;` は「PLAYER_SPEEDという名前の箱に4.0という数字を入れる」という意味です。この箱の中身を書き換えるだけで、ゲームの調整ができます。

メソッド (関数)



「ひとまとまりの処理」に名前をつけたものです。 `move()` は「動く」という処理、 `draw()` は「描画する」という処理。名前を見れば、何をする処理なのか大体わかるようになっています。

継承 (けいしょう)

既存の設計図をベースにして、新しい設計図を作ることです。 `class PlayerBullet extends Bullet` は「Bulletという基本の弾の設計図をベースに、PlayerBulletという弾を作る」という意味です。共通する部分（動き方など）を何度も書かずに済むための仕組みです。

全部覚えなくてもOK

この5つを完璧に理解していなくても、第4章のレシピ通りに数値や1～2行を書き換えるだけであれば問題なく改造できます。まずは触ってみましょう。

3 ファイルの地図

ゲームのコードは役割ごとに「セクション」に分かれています（1ファイル版では `// [SECTION] ∞` というコメントで区切られています。エディタの検索機能（Ctrl+F / Cmd+F）でセクション名を検索すると、すぐにその場所へジャンプできます）。

セクション名	カテゴリ	役割・こんな時に開く
Config	設定	ゲームの強さ・速さなど「数値」をまとめた場所。一番よく開くことになるセクション。
GameManager	全体管理	ゲーム全体の進行役。敵やプレイヤーの初期設定（最初に何を持たせるか）もここ。
CollisionManager	全体管理	「弾が当たったらどうなるか」を判定する場所。通常は触らなくてOK。
Player	プレイヤー	プレイヤーの移動・入力・ライフの管理。
Enemy	敵	敵の位置と、使う攻撃パターンの管理。
Bullet	弾	すべての弾に共通する基本設計図（自分で触ることは少ない）。
PlayerBullet / EnemyBullet	弾	プレイヤー弾・敵弾それぞれの色を決めている場所。
Weapon	武器	すべての武器に共通する基本設計図。
StraightShot	武器	プレイヤーの通常弾（自動で真上に飛ぶ攻撃）。
MeleeAttack	武器	プレイヤーの近接攻撃（周囲の敵弾を消す攻撃）。
BulletPattern	敵	すべての弾幕パターンに共通する基本設計図。
RadialPattern	敵	敵の「放射弾」の設定。弾数・速さ・間隔はConfigにある。
AimPattern	敵	敵の「自機狙い弾」の設定。
Effect	演出	

弾が相殺されたときの演出の呼び出し口（現在は中身が空）。

DrawUtils

表示

円を描く共通処理。見た目を画像に差し替えるときはここが起点になる。

DanmakuGame（メイン
部分）

全体管理

画面サイズの指定やキー入力の受付。

迷ったらこの表に戻ってきてください

「何かを変えたい」と思ったら、まずこの表で担当セクションを探し、ファイル内をそのセクション名で検索するのが一番の近道です。

目的別に「どこを」「どう」変えれば良いかを、実際のコードとあわせて紹介します。

4-1 ゲームバランスを調整する

1 敵の攻撃頻度を変える（放射弾）

やりたいこと： 敵の弾をもっと激しく／穏やかに撃たせたい

開く場所： `Config` セクション内、`ENEMY_RADIAL_INTERVAL`

```
static final int ENEMY_RADIAL_INTERVAL = 40; // 発射間隔（フレーム数）
```

この数字は「何フレームごとに1回発射するか」を表しています。60フレーム＝約1秒なので、40は「約0.67秒に1回」という意味になります。

変更前： 40 （やや控えめ）



変更後の例： 20 （倍の速さ・難しくなる）

ポイント

数字を小さくすると攻撃が激しくなり、大きくすると穏やかになります。

2 敵の弾の数・速さを変える（放射弾）

開く場所： `Config` セクション内

```
static final float ENEMY_RADIAL_BULLET_SPEED = 3.0; // 弾速
static final int   ENEMY_RADIAL_COUNT        = 12;  // 一度に放射する弾数
```

`ENEMY_RADIAL_BULLET_SPEED` を大きくすると弾が速く、`ENEMY_RADIAL_COUNT` を増やすと一度に飛んでくる弾の数が増えます。

注意

弾数を極端に増やす（例：100発）と処理が重くなる場合があります。まずは少しずつ調整してください。

3 自機狙い弾の頻度・速さを変える

開く場所： `Config` セクション内

```
static final float ENEMY_AIM_BULLET_SPEED = 4.0;
static final int   ENEMY_AIM_INTERVAL     = 50; // 発射間隔（フレーム数）
```

放射弾と同じ考え方です。`ENEMY_AIM_INTERVAL` を小さくすると狙い弾の頻度が上がります。

4 プレイヤーの移動速度を変える

開く場所: **Config** セクション内

```
static final float PLAYER_SPEED      = 4.0; // 通常移動速度
static final float PLAYER_SLOW_SPEED = 2.0; // Shift押下時の低速移動速度
```

それぞれ独立した数字なので、「普段は速いけどShift中はかなり遅くする」といった調整も可能です。

5 プレイヤーのライフを増減する

開く場所: **Config** セクション内

```
static final int  PLAYER_LIFE_MAX = 10; // 初期ライフ
```

難易度を上げたい → 5 など小さく

/

難易度を下げたい → 20 など大きく

6 プレイヤーの当たり判定の大きさを変える

開く場所: **Config** セクション内

```
static final float PLAYER_RADIUS = 6.0; // プレイヤーの当たり判定半径
```

弾幕ゲームでは、この当たり判定を見た目より少し小さくすることで「避けやすさ」を調整するのが定石です。

7 プレイヤーの通常弾の速さ・発射間隔を変える

開く場所: `Config` セクション内

```
static final float PLAYER_BULLET_SPEED    = 8.0;  
static final int   STRAIGHT_SHOT_INTERVAL = 10; // 発射間隔（フレーム数）
```

`STRAIGHT_SHOT_INTERVAL` を小さくすると、プレイヤーの攻撃頻度（連射力）が上がります。

8 近接攻撃の範囲・持続時間・発動間隔を変える

開く場所: `Config` セクション内

```
static final float MELEE_RADIUS    = 40.0; // 近接攻撃の有効半径  
static final int   MELEE_DURATION = 12;   // 有効時間（フレーム数）  
static final int   MELEE_INTERVAL = 30;   // 発動間隔（フレーム数）
```

`MELEE_RADIUS` を大きくすると1回で消せる敵弾の範囲が広がります。

9 制限時間を変える

開く場所: `Config` セクション内

```
static final int TIME_LIMIT_FRAMES = 60 * 60; // 制限時間（60秒 × 60FPS）
```

`60 * 60` の後ろの60を `90` にすれば90秒に、`30` にすれば30秒になります（60FPS前提なので「秒数 × 60」で計算します）。

4-2 見た目を変える

10 弾・プレイヤー・敵の色を変える

開く場所： `PlayerBullet` / `EnemyBullet` / `Player` / `Enemy` セクション

例えば敵弾の色を変えるには、`EnemyBullet` セクションの以下を書き換えます。

```
color getColor() {  
    return color(255, 80, 80); // 赤 (R, G, B の順)  
}
```

`color(赤, 緑, 青)` で、それぞれ0~255の数字で指定します。

紫にしたい → `color(180, 80, 220)`

黄色にしたい → `color(255, 220, 60)`

プレイヤーや敵の色は `Player` / `Enemy` セクションの `drawShape()` の中にある `drawCircle(pos, radius, color(...))` の色部分を同様に書き換えます。

11 弾や近接攻撃の大きさを変える

開く場所： `Config` セクション内

```
static final float PLAYER_BULLET_RADIUS    = 4.0;  
static final float ENEMY_RADIAL_BULLET_RADIUS = 6.0;  
static final float ENEMY_AIM_BULLET_RADIUS  = 6.0;
```

見た目のサイズと当たり判定のサイズが同じ変数で管理されているので、大きくすると「見た目も判定も」大きくなる点に注意してください。

12 画面サイズを変える

開く場所：メイン部分（1ファイル版の先頭）にある `setup()`

```
void setup() {  
  size(480, 640); // 横, 縦 (ピクセル)  
  gameManager = new GameManager();  
}
```

注意

画面サイズを変えると、敵やプレイヤーの初期位置（画面中央・下部など）も自動的にそれに合わせて計算されるので、位置がおかしくなることは基本ありません。ただし極端に小さくすると敵と自機が近すぎて避けにくくなります。

4-3 少しレベルアップ：新しい要素を追加する

13 新しい弾幕パターンを追加する

やりたいこと： 今ある「放射弾」「自機狙い弾」以外の新しい攻撃パターンを敵に持たせたい

手本にする場所： `RadialPattern` セクション（コピー元にする）

- 1 `RadialPattern` セクションのコードを丸ごとコピーします。
- 2 コピーしたコードの中の `class RadialPattern` という部分を、`class SpiralPattern` のような新しい名前に変更します（3箇所ある「`RadialPattern`」を全部同じ名前に揃えてください）。
- 3 `fire()` メソッドの中身を書き換えて、弾の飛ぶ方向・速さを好きに変更します。
- 4 `Config` セクションに、この新パターン専用の数値（間隔・速さなど）を追加します。
- 5 `GameManager` セクションの `setupEnemy()` の中に、次の1行を追加します。

```
enemy.patterns.add(new SpiralPattern());
```

ポイント

これだけで新しい攻撃パターンがゲームに反映されます。他のクラス（`Player`・`CollisionManager`など）は一切変更する必要がありません。これが「役割ごとに部品を分けておく」ことの一番のメリットです。

14 新しい武器を追加する

手本にする場所: `StraightShot` セクション (コピー元にする)

- 1 `StraightShot` セクションを丸ごとコピーします。
- 2 `class StraightShot` を `class SideShot` のような新しい名前に変更します (複数箇所を統一)。
- 3 `activate()` メソッドの中身を書き換えて、弾の発射方向などを変更します。
- 4 `GameManager` セクションの `setupPlayer()` に1行追加します。

```
player.weapons.add(new SideShot());
```

これでプレイヤーが複数の武器を同時に持つ状態になります。

数値をConfigの外に直接書かない

例えば `RadialPattern` の中に `speed = 5.0;` のように直接数値を書いてしまうと、「どこに何の数値があるか」が分からなくなり、後で調整するのが大変になります。数値は必ず `Config` セクションに追加し、そこから読み込むようにしてください。

波かっこ { } の対応を崩さない

このコードは `{` と `}` のペアで「ここからここまでが1つのまとまり」を表しています。コピー＆ペーストで一部だけ消してしまうと、ペアが崩れてエラーになります。迷ったら、変更前の状態と見比べて `{` の数と `}` の数が同じか確認してください。

セミコロン ; を忘れない

`float speed = 5.0` のように行の最後に `;` がないとエラーになります。数値だけを書き換えるときは、その行の末尾の `;` を誤って消さないよう注意してください。

クラス名の重複に注意

レシピ13・14のように既存クラスをコピーして新しいクラスを作るとき、クラス名を変更し忘れると「同じ名前のクラスが2つある」というエラーになります。コピーしたら**真っ先に**クラス名を変更する癖をつけてください。

全角文字の混入に注意

日本語入力中に、うっかり `;`（全角セミコロン）や `( )`（全角カッコ）、全角スペースを混ぜてしまうことがあります。見た目はそっくりですが、プログラムとしては全く別の文字として扱われ、エラーの原因になります。コメント（`//` より右側の日本語の説明文）以外の部分は、半角英数字で入力するようにしてください。

abstract がついたクラス・メソッドは直接使えない

`Bullet` ・ `Weapon` ・ `BulletPattern` には `abstract` という印がついています。これは「これ自体は設計図のひな形であり、そのままでは実物を作れない」という意味です。必ず `PlayerBullet` のように、これらを継承した具体的なクラスを経由して使ってください（普段の改造では、この印自体を触る必要はありません）。

6 困ったときは（エラー対処）

Processingで実行（▶）したときに赤字でエラーが表示されたら、慌てずに以下を確認してください。多くの場合、原因は単純な入力ミスです。

「Unexpected token」「missing ';'」などが出る

変更した行の末尾に `;` が付いているか確認してください。

直し方：行の最後にセミコロンを追加する。

「missing a closing '}'」が出る

波かっこ `{ }` の数が合っていません。直前に編集した箇所で、うっかり `}` を消してしまっていないか確認してください。

直し方：編集前のファイルと見比べて、かっこの数を揃える。

「cannot find anything named "〇〇"」が出る

変数名やクラス名のスペルミス（大文字・小文字の違いも含む）が疑われます。

× `Config.PLAYER_SPEED` のように、名前は1文字でも違うと「見つからない」扱いになります。

直し方： `Config` セクション内の実際の変数名をコピーして使う。

「〇〇 is already defined」が出る（クラスが重複）

レシピ13・14のようにクラスをコピーした際、クラス名を変え忘れていました。

直し方：新しく作ったクラスの名前を、元のクラスと違う名前に変更する。

見た目上は正しいのにエラーになる

日本語入力モードのまま記号（カッコ・セミコロン・スペースなど）を打ち込んでしまい、全角文字が紛れ込んでいる可能性があります。

直し方：該当行を一度すべて削除して、半角入力モードで打ち直す。

それでも直らないときは

- 変更した行を1つずつ元に戻し、どこからエラーが出始めたかを特定しましょう。
- エラーメッセージの右側にある行番号（例：「line 42」）を手がかりに、その行の周辺を確認しましょう。
- 1人で抱え込まず、チームメンバーに変更内容を見せて一緒に確認しましょう。

クラス (class)

ものの設計図。例： `Enemy` は敵の設計図。

インスタンス

設計図（クラス）から実際に作られた実物。 `new` という命令で作る。

変数

値を入れておく箱。名前と中身（数値や文字列など）を持つ。

メソッド (関数)

ひとまとまりの処理に名前をつけたもの。 `move()` や `draw()` など。

継承 (extends)

既存のクラスをベースに新しいクラスを作ること。共通部分を使い回せる。

abstract (抽象)

「これ自体は実物を作れない、ひな形専用のクラス／メソッド」という印。

ArrayList (配列のようなもの)

複数のモノをまとめて入れておける箱。敵弾やプレイヤー弾は全部この中で管理されている。

フレーム / FPS

ゲームは1秒間に60回（60FPS）画面を描き直している。「間隔」の数値は、この回数を基準にしている。

PVector

位置や速度など、 $x \cdot y$ の2つの数字をセットで扱うための入れ物。

コメント (// から始まる行)

プログラムとしては実行されない、人間向けのメモ書き。日本語の説明はここに書かれている。

弾幕ゲーム 改造マニュアル / このドキュメントは今後のプロジェクトの成長に合わせて更新してください。